

Reproducibility guide

Repository source: REPRODUCIBILITY.md

Contents

Reproduction levels	1
Tested environments	2
Construction environment	2
User-machine replay	2
Continuous integration	2
Fresh source-tree procedure	2
Python exact path and audits	3
Independent base-R audit	3
Complete language replay	4
Paper build	4
Markdown/PDF documentation build	4
PDF preflight	4
Clean-tree check	5
Disk and network requirements	5
Reproducibility is not peer review	5

Package status: public-release construction checkpoint, not yet tagged or released **Prepared:** September 19, 2026
Main certificate schema: minvol.universal_contact_bound.v1

This guide explains how to reproduce the exact certificate, the independent Python and base-R audits, the adversarial mutation tests, the paper/certificate reconciliation, the manuscript build, and the paired Markdown/PDF documentation.

The proposed theorem represented by the package is

$$\text{Vol}(K) \geq \frac{26220940089713\pi}{20000000000000} d^3 = 0.41187756378030227424817892598 \dots d^3$$

for every convex body $K \subset \mathbb{R}^3$ of constant width $d > 0$.

Reproduction levels

The repository distinguishes four levels of reproduction.

Level	What is reproduced	Required tools	Interpretation
Exact certificate	All theorem-bearing finite inequalities and byte-identical derived certificate files	Python and pinned NumPy	Finite proof authority
Independent audits	Binary64 Python and base-R reconstructions	Python/NumPy and base R	Independent numerical agreement, not proof authority
Paper reconciliation	The theorem coefficient, split radius, lower-row table, and paper-facing constants	Python	Prevents silent drift between paper and certificate

Level	What is reproduced	Required tools	Interpretation
Document build	LaTeX paper and every Markdown-derived PDF	LaTeX, Pandoc, XeLaTeX; Poppler for full preflight	Presentation and release integrity

Tested environments

Construction environment

The archived construction transcript records:

```
Linux x86_64
Python 3.13.5
NumPy 2.3.5
Rscript unavailable
```

In that environment, the exact verifier, binary64 audit, ten reconstruction tests, seven adversarial mutation tests, paper reconciliation, manuscript build, Markdown/PDF build, and Poppler PDF preflight passed.

User-machine replay

The named author reproduced the standalone package on macOS with:

```
Python 3.14
NumPy 2.3.5
base R available
```

The exact verifier, binary64 audit, ten reconstruction tests, and base-R audit passed. The posted transcript did not record the exact R version, so this document does not infer one. The local Mac did not have Poppler installed; full `pdftotext`, `pdffonts`, and `pdfinfo` preflight is therefore assigned to the GitHub Actions document job, matching the workflow used for the first public MinVol release.

Continuous integration

The workflow `.github/workflows/verification.yml` is configured for:

- Ubuntu 24.04;
- Python 3.13;
- NumPy 2.3.5 from `requirements.txt`;
- R 4.6.1, using base R only;
- Pandoc 3.10.1;
- XeLaTeX and `latexmk`; and
- Poppler `pdfinfo`, `pdffonts`, and `pdftotext`.

A green hosted workflow is a release gate, but it does not replace mathematical review.

Fresh source-tree procedure

Until a public remote and immutable tag exist, reproduce from the exact source archive or bundle supplied for review. After release, this section will be updated with the canonical repository URL and tag.

Enter the repository root:

```
cd minvol-degree3-contact-bound
```

Verify that the generated checksum manifests are current:

```
python3 refresh_checksums.py --check
```

Verify the root manifest:

```
shasum -a 256 -c SHA256SUMS.txt
```

Verify the certificate manifest:

```
cd certificate
```

```
shasum -a 256 -c SHA256SUMS.txt
```

Verify the paper manifest:

```
cd ../paper
```

```
shasum -a 256 -c SHA256SUMS.txt
```

Return to the root:

```
cd ..
```

Python exact path and audits

Create an isolated environment:

```
python3 -m venv .venv
```

Activate it on a POSIX shell:

```
./venv/bin/activate
```

Install the pinned dependency:

```
python3 -m pip install -r requirements.txt
```

Run the exact verifier, independent binary64 audit, ten reconstruction tests, seven mutation tests, exact-constants appendix check, release-document check, and paper reconciliation:

```
./run_python_checks.sh
```

A successful run contains all of these markers:

```
MINVOL UNIVERSAL CONTACT-BOUND CERTIFICATE: EXACT
PROPOSED UNIVERSAL LOWER BOUND ABOVE 0.4118775: EXACTLY CERTIFIED BY THIS PACKAGE
MINVOL UNIVERSAL CONTACT-BOUND BINARY64 AUDIT: PASS
Ran 10 tests
OK
Ran 7 tests
OK
MINVOL ADVERSARIAL MUTATION SUITE: PASS
MINVOL EXACT-CONSTANTS APPENDIX: CURRENT
MINVOL RELEASE-DOCUMENT CHECK: PASS
MINVOL PAPER-CERTIFICATE RECONCILIATION: PASS
```

On tested machines, the complete Python path usually finishes in under one minute. Runtime is informative only; a slower run is not a failure.

Independent base-R audit

The R program uses base R only:

```
./run_r_checks.sh
```

The required marker is:

```
MINVOL UNIVERSAL CONTACT-BOUND BASE-R AUDIT: PASS
```

To deliberately refresh the archived transcript:

```
./run_r_checks.sh 2>&1 | tee certificate/transcripts/base_r_audit.txt
```

After changing a protected transcript, refresh both generated manifests:

```
python3 refresh_checksums.py
```

Then rerun all checksum checks. A transcript should never be edited to add a pass marker that was not produced by the program.

Complete language replay

When both Python and R are available:

```
./run_all.sh
```

The final marker is:

```
MINVOL UNIVERSAL CONTACT-BOUND REPLAY: PASS
```

Paper build

The committed paper PDF is a reference artifact. A local build is written to `paper/build/` and does not overwrite the reference PDF:

```
./build_paper.sh
```

Required final markers:

```
MINVOL PAPER-CERTIFICATE RECONCILIATION: PASS  
MINVOL PAPER BUILD: PASS
```

A locally rebuilt PDF need not have the same SHA-256 as the committed PDF on a different TeX installation. Mathematical content, page validity, embedded fonts, extractable text, and reconciliation with the certificate are the portable requirements.

Markdown/PDF documentation build

Build all Markdown sources into a local mirror under `build/markdown/`:

```
./build_markdown_pdfs.sh
```

Verify the committed pairs:

```
python3 ci/verify_doc_pairs.py --pdf-root rendered/markdown
```

Verify the local rebuild pairs:

```
python3 ci/verify_doc_pairs.py --pdf-root build/markdown
```

The committed reference PDFs are in `rendered/markdown/`; Markdown remains the editable source.

PDF preflight

On a system with Poppler installed, preflight the committed artifacts:

```
./ci/preflight_pdfs.sh rendered/markdown paper/contact_geometric_bound.pdf
```

Preflight the local rebuilds:

```
./ci/preflight_pdfs.sh build/markdown paper/build/contact_geometric_bound.pdf
```

If Poppler is not available locally, record that fact and rely on the hosted document job. Do not weaken or delete the preflight script merely to make a local command return success.

Clean-tree check

After all replay steps and deliberate builds:

```
git status --short
```

A release-candidate replay should leave the tracked source tree clean. Local outputs under ignored `build/`, `.venv/`, and `ci-artifacts/` directories do not count as tracked changes.

Disk and network requirements

The checked proof objects occupy only a few megabytes. The largest cost is the local TeX/Pandoc toolchain rather than the certificate itself. The exact verifier requires no network access after the repository and Python dependency are present. Continuous integration may access package mirrors only to install its declared tools.

Reproducibility is not peer review

Successful replay shows that the supplied programs deterministically accept the supplied exact objects and that two numerical implementations agree. It does not independently prove the geometric reductions in the manuscript, establish novelty, identify a minimizer, or replace expert mathematical review. See [certificate/TRUST_BOUNDARY.md](#) and [certificate/CLAIMS_AND_LIMITATIONS.md](#).